



University of
Southern Denmark

2nd Semester Project - HeatItOn

Heat Production Optimization

Group 8

Adela Pastekova – adpas25@student.sdu.dk

Alex Zalanyi – alzal25@student.sdu.dk

Filip Tichy – fitic25@student.sdu.dk

Marija Savkina – masav25@student.sdu.dk

Melana Ohrimeca – meohr25@student.sdu.dk

Zoltan Suveges – zosuv25@student.sdu.dk

May 14, 2026

Contents

Abstract	5
0.1 Abstract	5
1 Introduction	1
1.1 Introduction	1
1.2 Functional Requirements	3
1.3 Non-Functional Requirements	7
1.4 User Requirements	8
1.5 System Requirements	8
2 Scrum Implementation	10
2.1 Scrum Implementation	10
3 Sprints	11
3.1 Sprints	11
4 Solution(Technical Aspect)	12
4.1 Verb-noun analysis	12
4.2 Classes and Methods Mapping	14
4.3 System Design	16
4.3.1 UML Diagram - Class Diagram	16
4.3.2 UML Diagram - Package Diagram	18
4.3.3 UML Diagram - Activity Diagram (Add asset)	20
4.3.4 UML Diagram - Activity Diagram (Import Sources)	21
4.3.5 UML Diagram - Activity Diagram (Delete Asset)	22
4.3.6 UML Diagram - Activity Diagram (Optimize)	23
4.3.7 UML Diagram - Sequence Diagram	24
4.4 System Technical Architecture	24
5 Implementation and Technical Realization	26
5.1 Frontend Implementation	26
5.1.1 Midterm Version	27
5.1.2 Final Version	29
5.2 Backend Implementation	29

5.2.1	Midterm Version	29
5.2.2	Final Version	29
5.3	Data Management	29
5.3.1	Midterm Version	29
5.3.2	Final Version	29
6	Discussion and Reflection	30
6.1	Discussion and Reflection	30
7	Conclusion	31
7.1	Conclusion	31
8	References	32
8.1	References	32
9	AI Usage	33
9.1	AI Usage	33
A	Appendix	34
A.1	Appendices	34

List of Figures

1.1	System Architecture Diagram	2
4.1	Combined Frontend and Backend Class Diagram (In-Progress)	16
4.2	Package UML Diagram - Five main packages of the system. .	18
4.3	Add Asset Activity UML Diagram - Three parts and their actions for adding a new Asset.	20
4.4	Import Sources Activity UML Diagram - Three parts and their actions for importing Sources from a CSV file.	21
4.5	Delete Asset Activity UML Diagram - Three parts and their actions for deleting Assets.	22
4.6	Optimization Activity UML Diagram - Three parts and their actions for deleting Assets.	23
5.1	Assets tab from the demo version	28
5.2	Results tab from the demo version	28

List of Tables

[

Abstract]Abstract

0.1 Abstract

This is our Abstract.

Chapter 1

Introduction

1.1 Introduction

Problem Statement

HeatItOn is a utility in the city of Heating that supplies heat to approximately 1600 buildings. HeatItOn currently plans heat production manually, which makes it hard to always choose the cheapest combination of production units. The problem is therefore to develop an optimization system that can automatically create heat production schedules that always meet the heat demand while minimizing total costs and maximizing profit from electricity production/consumption.

Problem Analysis

HeatItOn is a utility in the city of Heating that supplies heat to approximately 1600 buildings through a district heating network. Heat is produced by traditional heat-only boilers and by units that both produce heat and produce or consume electricity (CHPs). Most of the heat comes from cogeneration plants that use different fuels such as waste, gas, coal, and biomass.

District heating works by pumping hot water from a plant through insulated pipes to buildings, where it is used for space heating and hot tap water, then cooled water then flows back to the plant to be reheated.

Today, HeatItOn plans its heat production manually. This makes it difficult to choose the best combination of units and to react to changing electricity prices in a systematic way. Our task is to develop software that optimizes heat production and creates heat schedules for all available units with the lowest possible costs and the highest possible profit from the electricity market.

Our system will consist of 5 main modules:

- Optimizer (OPT)
- Data Visualization (DV)
- Asset Manager (AM)
- Source Data Manager (SDM)
- Result Data Manager (RDM)

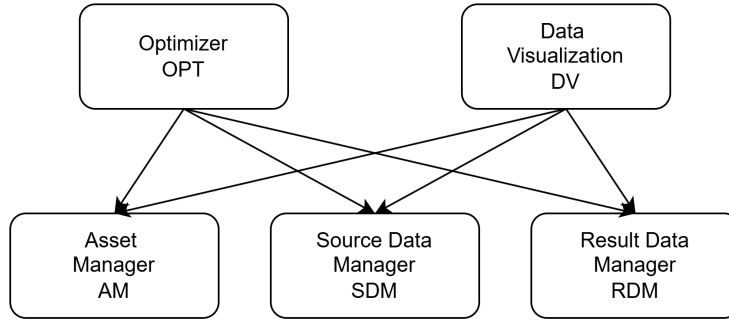


Figure 1.1: System Architecture Diagram

Optimizer (OPT):

The OPT considers all the available production units and calculates the best economical schedule of the heat production while securing heat availability to all buildings at any time. All units have a specific production cost, expressed in DKK per MWh produced heat. The source of this information is excel file called 2026 Heat Production Optimization - Danfoss Deliveries

Data Visualization (DV):

The DV is responsible for a graphical presentation of important source and result data.

Asset Manager (AM):

The AM is a repository for static system information and is responsible for making this information available for other modules. Through the AM, other modules can retrieve heating grid information such as name and a graphical representation of the grid.

Source Data Manager (SDM):

The SDM is a repository for dynamic system information. The SDM can hold its data in local CSV files. The SDM provides time series for the heat demand and electricity prices. Both the heat demand and the electricity prices are given with a resolution of 1 hour.

Result Data Manager (RDM):

The RDM is a repository for optimization results

Scenarios

Scenario 1 (Heat only)

Scenario 1 uses a single district heating network with three gas boilers and one oil boiler. The gas boilers are cheaper, while the oil boiler has higher production costs and is mainly used in peak demand periods. All boilers have constant production costs that do not depend on the hour of operation. Our strategy is to run the cheapest gas boiler first and then add production from the more expensive boilers if needed to meet the heat demand. We will also have to choose a period of 30 to 60 hours when one of the gas boiler will be down due to maintenance.

Scenario 2 (Heat and electricity)

In Scenario 2, we work with two gas boilers (GB1, GB3), one gas motor (GM1), and one electric boiler (EB1). The gas motor produces electricity (which we can sell), while the electric boiler consumes electricity (which we must buy). When electricity prices are high we will run the gas motor because selling electricity generates extra income. When electricity prices are low we will run the electric boiler because electricity is cheap to buy.

To decide which units to use in each hour, we will calculate the net production cost for each unit and compare them.

In this project we will use data delivered by Danfoss, including:

- Graphical representation of the district heating network
- Configuration parameters for each production unit (image, produced heat, produced/consumed electricity, primary energy consumption, production costs, CO₂ emissions)
- Heat demand and electricity prices for a 14-day winter period with 1-hour resolution
- Heat demand and electricity prices for a 14-day summer period with 1-hour resolution

If there is time left, we can extend the project by using other visualization libraries, storing data in a database, enabling or disabling units directly from the UI, running alternative configurations in Scenario 2, and implementing APIs and an API client for reading electricity prices from Energinet.dk.

1.2 Functional Requirements

Scenario 1

1. Source Data Manager (SDM) & Result Data Manager (RDM)

Components that handle where the data comes from and where it goes.

- The user should be able to receive input data.
- The user should be able to retrieve input data.
- The user should be able to output data. [**Database integration**]

2. Asset Manager (AM)

Component that keeps track of the physical boilers and their constraints.

- The user should be able to upload assets from csv file.
- The user should be able to add asset.
- The user should be able to edit assets. [**Boiler definitions**]
- The user should restrict the maximum heat output of each unit as follows: GB1 to 3.0 MW, GB2 to 2.0 MW, GB3 to 4.0 MW, and OB1 to 6.0 MW. [**Capacity constraints**]

3. Optimizer (OPT)

- The user should be able to schedule the cheapest boilers first to fulfill the district's heat demand. [**Cost-based dispatch**]
- Based on production costs, the optimizer should prioritize boilers in the following strict order: GB1 (510 DKK), GB2 (540 DKK), GB3 (580 DKK), OB1 (690 DKK). [**Dispatch order**]
- The user should only use the more expensive oil boiler (OB1) to supplement heat during peak demand periods when the gas boilers cannot fulfil the total demand. [**Peak demand handling**]
- The user should be capable of optimizing units separately for both a winter period and a summer period. [**Seasonal optimization**]
- The optimizer should remove a designated gas boiler from the available pool of units during its scheduled maintenance period, ensuring it produces 0 MW. [**Maintenance logic**]

4. Data Visualization (DV) / User Interface

- The UI should validate that the user's selected maintenance duration is a minimum of 30 hours and a maximum of 60 hours. [**Maintenance constraints validation**]
- The UI should display a chart or graph that visually compares the total heat demand against the individual heat output contributions of GB1, GB2, GB3, and OB1 over time. [**Result graphing**]

- The UI should provide a way for the user to choose an obligatory maintenance period for one unit in both the summer and winter periods. **[Maintenance scheduling]**

Scenario 2

1. Source Data Manager (SDM) & Result Data Manager (RDM)

Component that handle a crucial data stream: electricity prices.

- The user should be able to retrieve hourly electricity prices data from data base.
- The user should be able to retrieve hourly heat demand data from the database. **[Market data retrieval]**

2. Asset Manager (AM)

The physical assets include units that interact with the power grid.

- The user should be able to upload assets from csv file. **[Unit registration]**
- The user should be able to limit the maximum heat output of each unit: GB1 to 3.0 MW, GB3 to 4.0 MW, GM1 to 5.3 MW, and EB1 to 6.0 MW. **[Heat capacity constraints]**
- The user should account for electricity production and consumption capacities, specifically noting that GM1 produces up to 3.9 MW of electricity, while EB1 consumes up to 6.0 MW (-6.0 MW). **[Electricity constraints]**

3. Optimizer (OPT)

- The user should be able to calculate the net production costs for the electricity-producing/consuming units based on the fluctuating electricity prices. **[Dynamic net cost calculation]**
- The optimizer should prioritize running the gas motor (GM1) during periods of high electricity prices. **[High-price optimization logic]**
- The optimizer should prioritize running the electric boiler (EB1) during periods of low electricity prices. **[Low-price optimization logic]**
- The optimizer should actively prevent the gas motor (GM1) from running during summer periods with negative electricity prices (to avoid paying for produced electricity).

- The optimizer should recognize the electric boiler as an extra income source during negative price hours. [**Negative price handling**]
- The optimizer should completely remove either GB1 or GM1 from the optimization calculations during its scheduled maintenance period. [**Maintenance logic**]

4. Data Visualization (DV) / User Interface

- The UI should allow the user to select an obligatory maintenance period for either GB1 or GM1, enforcing a duration between 30 and 60 hours. [**Maintenance scheduling**]
- The UI should display a graph comparing the total heat demand against the individual heat output contributions of GB1, GB3, GM1, and EB1 over time. [**Result graphing**]

General App Navigation & UI

Requirements that cover the flow of the application from the user's perspective.

- Upon launch, the application should display a home screen allowing the user to explicitly select between Scenario 1 and Scenario 2. [**Startup Page**]
- The user interface should include a persistent navigation element that allows the user to switch between Scenario 1 and Scenario 2 without needing to return to the initial home screen. [**Persistent navigation**]

System Architecture & Data Handling

Requirements that cover how the app manages data and how the 5 components talk to each other.

- The user should be able to use a database system to save and retrieve data. [**Database storage**]
- The Source Data Manager should implement an API client to directly read real-time electricity prices from the Energinet.dk web servers. [**External API integration**]

Advanced Configuration & Comparison

Requirements that give the user more control over the optimization process.

- The UI should provide a mechanism for the user to dynamically activate or deactivate individual heating units from being considered in the optimizer's calculations. [**Dynamic unit toggling**]
- The system should allow the user to run Scenario 2 under different configurations. [**Scenario 2 configuration comparison**]
- The Data Visualization module should compare the results of these different Scenario 2 configurations specifically in terms of generated revenue and total CO₂ emissions. [**Comparison metrics**]

1.3 Non-Functional Requirements

1. Performance & Efficiency

- The system should complete all optimization calculations within 3 seconds of the user clicking or "Optimize."
- The system should completely render the resulting graphs within 3 seconds of the user clicking "Run". [**Processing speed**]
- The system and database should be capable of efficiently loading, processing, and aggregating a full month of hourly data (approx. 720-744 data points) without UI freezing or lag. [**Data volume handling**]

2. Environment & Portability

- The application should be fully cross-platform compatible, executing reliably on Windows, macOS, and Linux operating systems. [**Cross-platform compatibility**]

3. Usability & Localization

- The entire user interface, including all menus, labels, and error messages, should be written in English. [**UI Language**]
- All financial values, production costs, and revenues should be calculated and displayed strictly using Danish Kroner (DKK). [**Currency localization**]
- The resulting visualization for a month of data should be highly readable, formatting the daily boiler usage as a stacked chart (matching the total heat demand on the Y-axis, and days of the month on the X-axis) alongside a daily cost graph. [**Visualization clarity**]

4. Reliability & Robustness

- The system should not crash if the Energinet.dk API is unresponsive or if the user loses internet connection. [**System stability**]
- In the case of an API failure, the system should handle the error by displaying a clear, understandable error feedback message or popup to the user. [**Exception handling**]

1.4 User Requirements

The system is made with a certain type of user in mind. They should have the following attributes for optimal user experience:

- Be able to read in English
- Be able to navigate a simple UI
- Have a basic understanding of the problem being solved by the system
- Be able to read and understand input/output data

1.5 System Requirements

The system is split into three parts: Front End, Back End and the Database. Both the Backend and the Database are run in a Linux Docker container. The Frontend is run locally on the user's machine. For a machine to run the project they need the following:

Software Specifications

Entire project: .NET 9.0

Backend: Docker Desktop (latest version)

Database: Docker Desktop with MySql Image (shall be installed automatically if missing)

Frontend: Required NuGet packages (all must be compatible with .NET 9.0):

- Avalonia.UI

Hardware Requirements

- WSL version 2.1.5 or later (for Windows)
- Any of the following OS's: Windows 10/11 (64 bit), MacOS, Linux (64 bit)
- 4GB of RAM
- Motherboard supporting hardware optimization
- 3GB of free space

Chapter 2

Scrum Implementation

2.1 Scrum Implementation

This is our Scrum Implementation.

Chapter 3

Sprints

3.1 Sprints

This is our Sprints.

Chapter 4

Solution(Technical Aspect)

This section will outline the process of creation of the project, from project description to final solution, also describing the technical architecture. The solution started off with going through the provided Project Description. From here important parts were selected and wrote into a compact version. This compact version was suitable for conducting the verb/noun analysis where class and method candidates emerged. The class candidates are able to be transformed into CRC cards outlining the responsibilities and collaborators of each class. The CRC cards are used to create UML diagrams(Class,Package,Activity and Sequence). The different diagrams later on were used for implementation.

4.1 Verb-noun analysis

Asset Manager (AM) should store its data in local files and supply other modules with it. Source Data Manager (SDM) should store dynamic data in csv files. Result Data Manager (RDM) should save result data in local CSV files. Optimizer (OPT) should calculate the best economical schedule of the heat production for a given period. It also secures heat availability to all buildings at any time. There are different units, each has a specific production cost, expressed in DKK per MWh produced heat. For heat only units there are no further expenses nor income. Electricity producing units should sell the produced electricity and have thereby an extra income depending on the electricity prices which can change from hour to hour. Electricity consuming units should buy the electricity and have thereby an extra expense, again depending on the electricity prices. Net production cost should be calculated differently for each unit type and compared to build priorities. The Data Visualization should display a graphical presentation of important source and result data.

Candidates for classes:

- Asset Manager
- Data
- Files
- Modules
- Source Data Manager
- Result Data Manager
- Optimizer
- Schedule
- Production
- Period
- Heat availability
- Buildings
- Time
- Electricity
- Units (GB1-EB1)
- Income
- Expenses
- Prices
- Net production cost
- Unit type
- Data Visualization
- Presentation
- Priorities

Candidates for methods:

- Store
- Supply
- Save
- Calculate
- Secures
- Has
- Sell
- Have
- Depending
- Change
- Buy
- Compare
- Display

4.2 Classes and Methods Mapping

Class Name	Associated Methods
AssetManager	store, supply
SourceDataManager	store (dynamic data), read CSV
ResultDataManager	save (results)
Optimizer	calculate (schedule), secure (heat availability), compare
Unit	sell (electricity), buy (electricity), calculate net production cost
DataVisualization	display (graphical data)

CRC Cards

Class: Asset Manager	
Responsibilities	Collaborators
• Store heater data	• Optimizer • Data Visualization • Unit

Class: Optimizer	
Responsibilities	Collaborators
• Calculates heating schedule	• Asset Manager • Source Data Manager • Result Data Manager

Class: Source Data Manager	
Responsibilities	Collaborators
• Stores market data	• Optimizer • Data Visualization

Class: Result Data Manager	
Responsibilities	Collaborators
• Save output data	• Data Visualization • Optimizer

Class: Data Visualization	
Responsibilities	Collaborators
• Display important data to the user	• Source Data Manager • Result Data Manager • Asset Manager

Class: Units	
Responsibilities	Collaborators
• Contains data for different heaters • Calculate net production costs	• Optimizer • Asset Manager

4.3 System Design

4.3.1 UML Diagram - Class Diagram

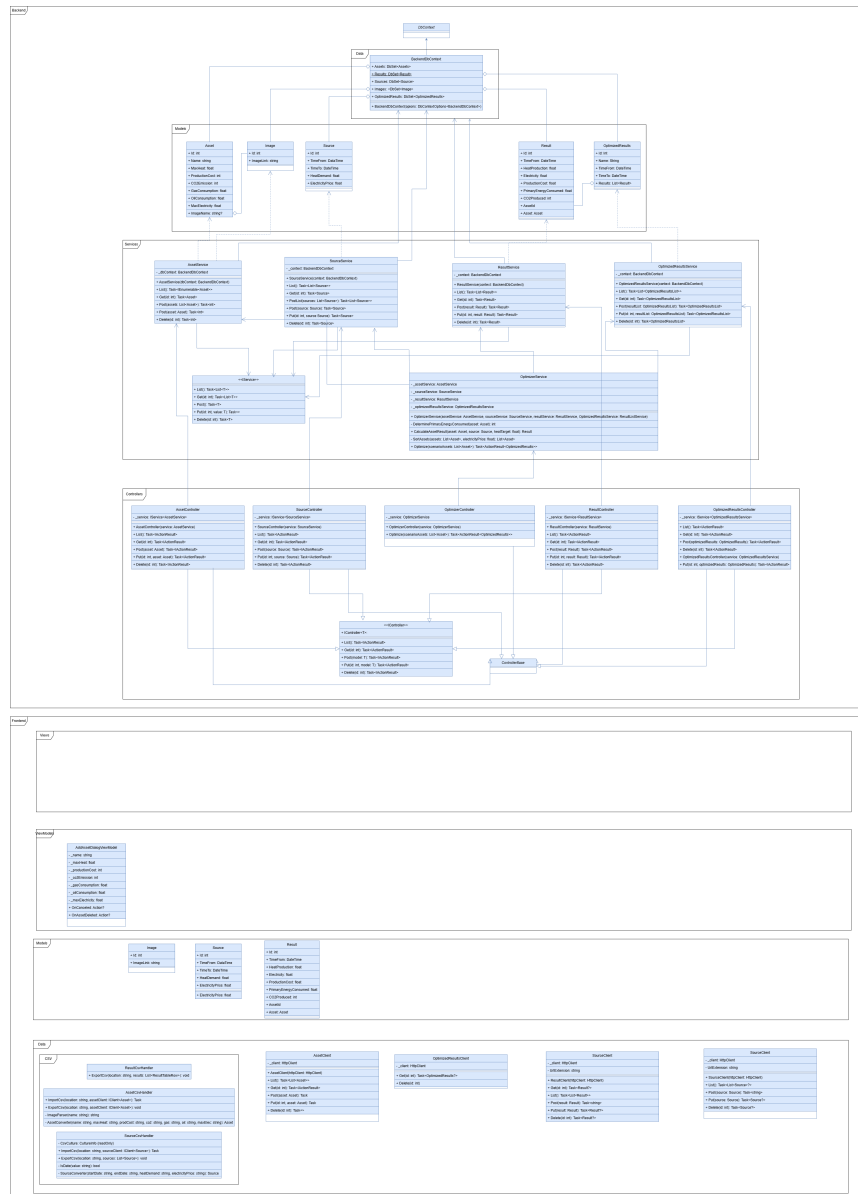


Figure 4.1: Combined Frontend and Backend Class Diagram (In-Progress)

The diagram (Figure 4.1) represents the unified system architecture. This is a preliminary version that integrates both the UI components (Frontend) and the logic/data services (Backend).

4.3.2 UML Diagram - Package Diagram

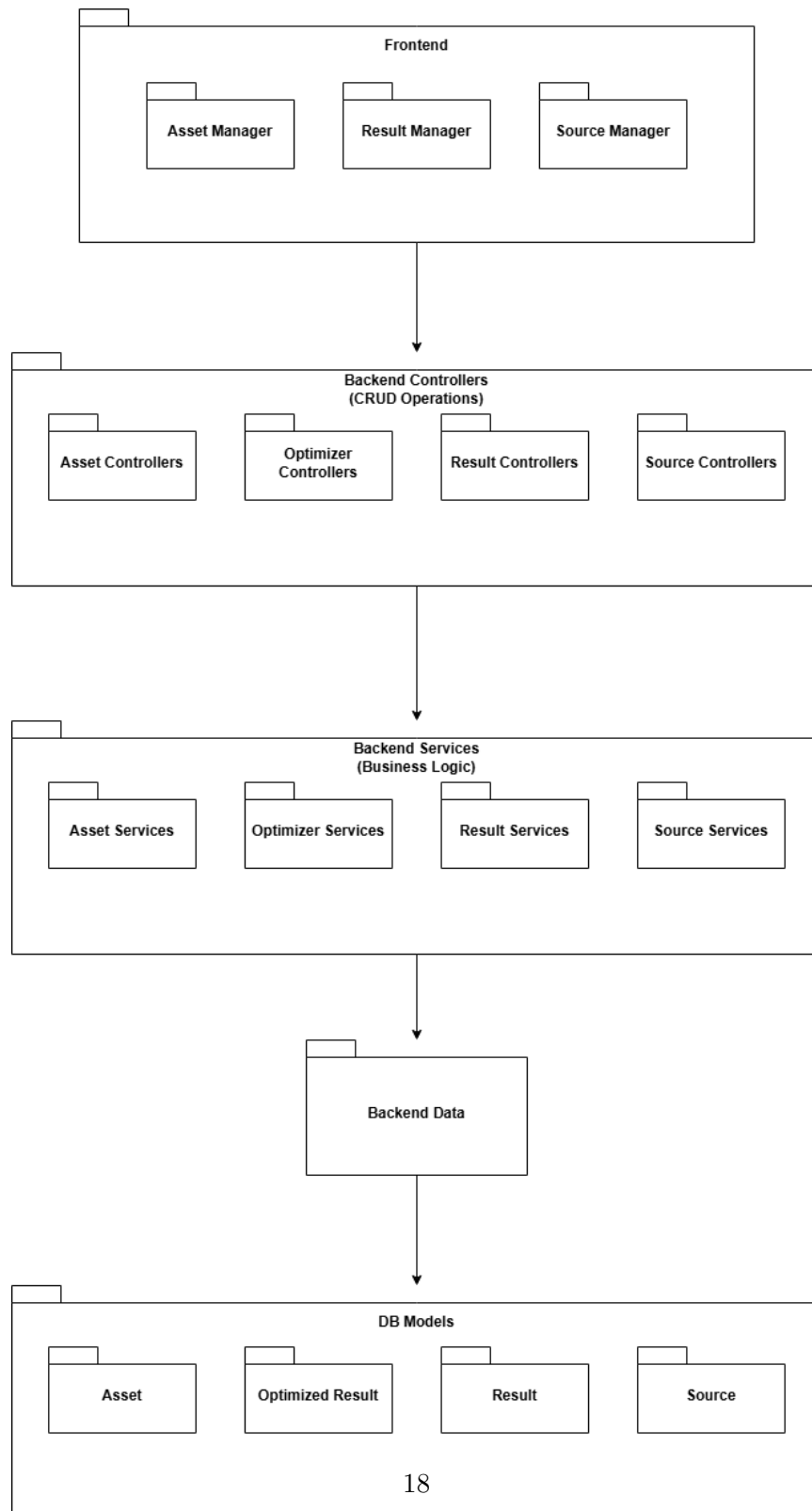


Figure 4.2: Package UML Diagram - Five main packages of the system.

The package diagram shown (Figure 4.2) simplified the system into five packages.

The first one is the Frontend package, containing the Asset, Result and Source Managers. This is the UI of the system, with what the user will interact. The Frontend is following the Model-View-ViewModel pattern, layering the different parts of each manager. This package connects with the next package through clients.

The second package, the Backend Controllers, contain the CRUD(Create, Read, Update, Delete) operations of each module. Through HTTP requests these can be accessed and will call the services in the next package.

Backend Services are the third package containing all the business logic (like the Optimization algorithm).

The services communicate with the Database for which the connection, the fourth package, is located inside the Data layer. The database is responsible for storing elements of the fifth package (Asset, Source, Result), the models.

This diagram shows a very simplified relationship with the most important parts of the overall system, giving an understanding of the overall workings and layers. Although it is very simple, it is a good starting point for understanding the entire architecture.

These packages, in the final solution, while expanded upon greatly, are the backbone of the implementation.

4.3.3 UML Diagram - Activity Diagram (Add asset)

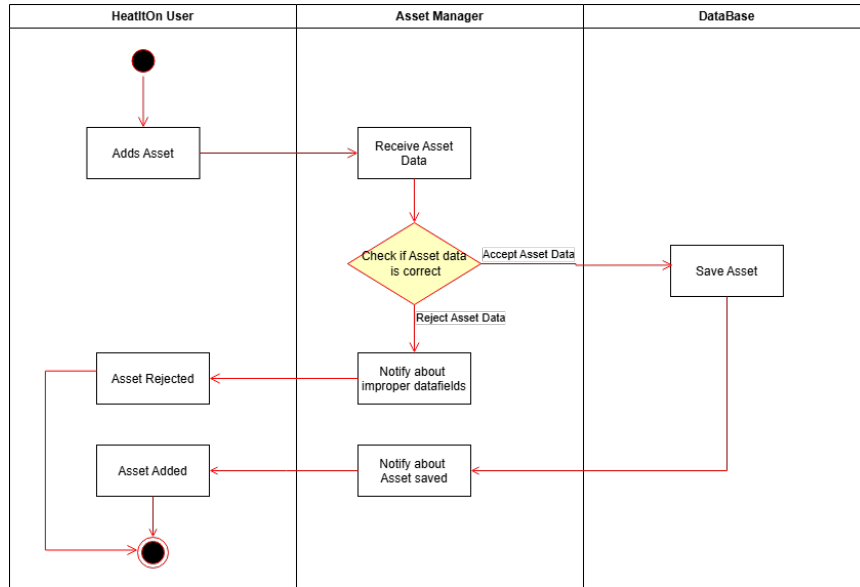


Figure 4.3: Add Asset Activity UML Diagram - Three parts and their actions for adding a new Asset.

The activity UML diagram (Figure 4.3) depicts the addition of a new asset by the user into the system.

The activity is initiated by the user when they want to add a new asset interacting with the Asset Manager. The filled out data is then verified by the Asset Manager. If the data was filled out correctly (like having a proper name or number fields having numbers) the Asset Manager will create the Asset and send it to the DataBase, who will save it. If the data was not filled out correctly, it will reject the received data. The activity ends with the user being notified of both cases, ending it. For a successful addition the Asset Manager will also display the new Asset instantly.

Adding an asset is started in the FrontEnd. The Frontend clients will call the Backend that will handle the communication with the Database where the Asset will be saved. The Frontend deals with user input and handling the data as well.

This diagram helps getting an idea about how the communication, for adding Assets, works. This activity is crucial to the solution, as every Asset needs to be stored in the Database for the optimization to work.

4.3.4 UML Diagram - Activity Diagram (Import Sources)

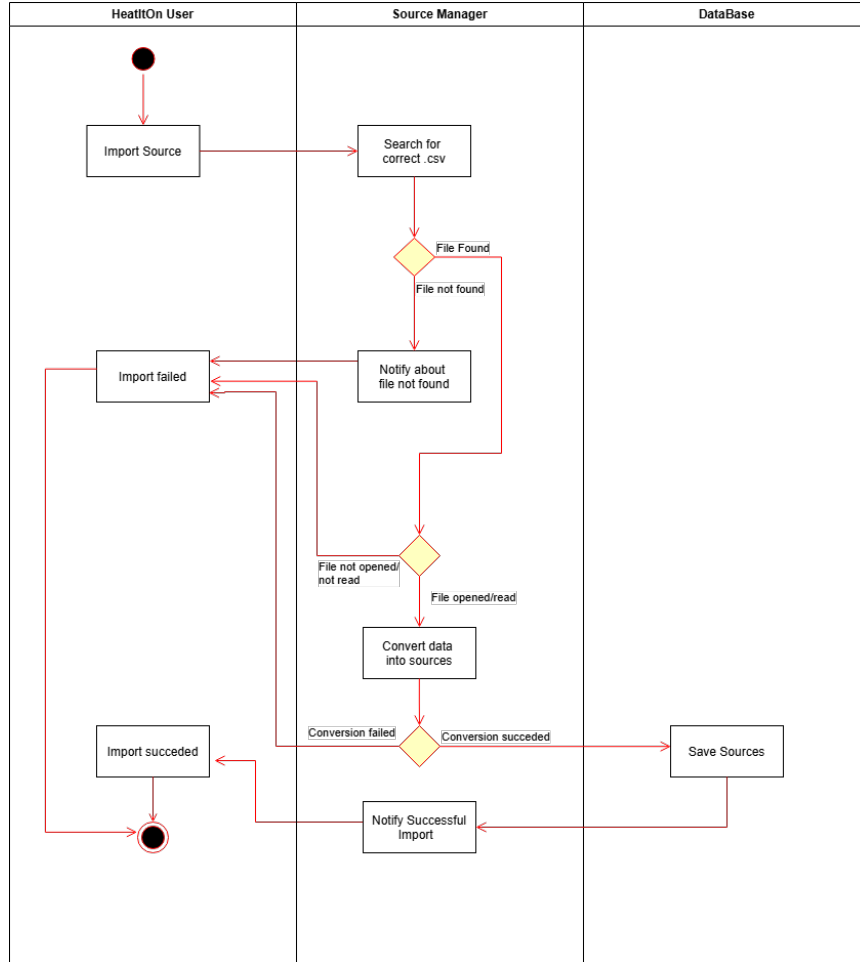


Figure 4.4: Import Sources Activity UML Diagram - Three parts and their actions for importing Sources from a CSV file.

The activity UML diagram (Figure 4.4) depicts the importing of one or multiple Sources stored in a Comma Separated Value(csv) file.

Importing is initiated by the user interacting with the Source Manager UI. The Source Manager then will try to find the requested csv file. If the file is found the Source Manager will try to open and read the data in the FrontEnd. After a successful opening and reading the data will be transformed into proper Source models and will be sent over through the Backend to the Database for storage, notifying the user. If the file is not found or if the file cannot be opened or read properly, like if the csv is not structured properly, the importing process fails and the user is notified. The activity is concluded by notifying the user of either a successful or failed

import, the former is visible by a new data set appearing in the Source Manager.

This helps getting an understanding of how importing works for sources (but not only), and the workings of the Source Manager.

The Source Manager and Sources are the other crucial part of the system, which without, the optimization process could not be run.

4.3.5 UML Diagram - Activity Diagram (Delete Asset)

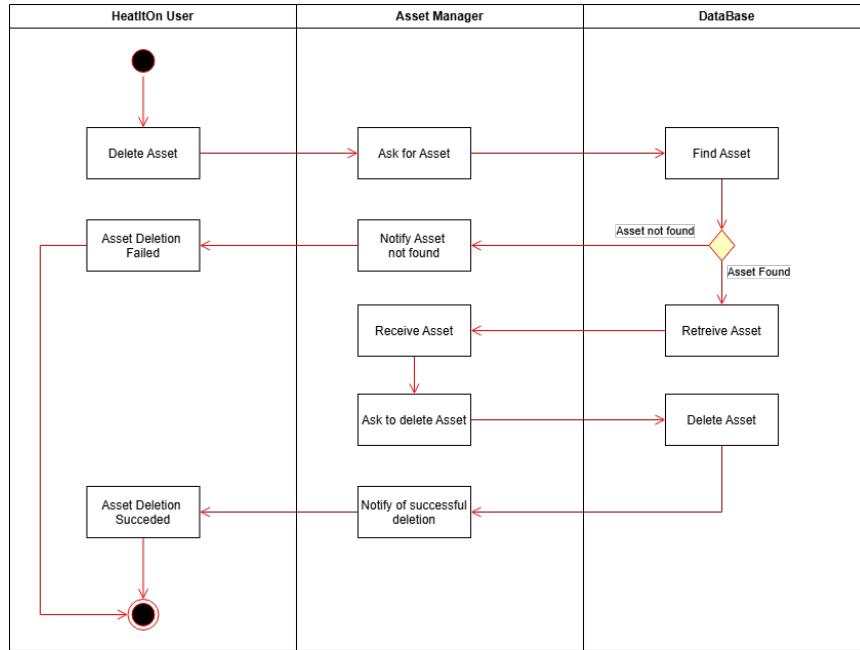


Figure 4.5: Delete Asset Activity UML Diagram - Three parts and their actions for deleting Assets.

The activity UML diagram (Figure 4.5) depicts the deletion of an Asset through the Asset Manager.

Deletion is initiated by the user through interacting with the Asset Manager UI. The Asset Manager will try to retrieve the Asset from the DataBase(through the Backend). If the DataBase finds the Asset it will return it to the Asset Manager. If the DataBase could not find the requested Asset, it will prompt the Asset Manager will notify the user ending the activity. If the Asset is found it will be retrieved to the Asset Manager who will request its deletion from the DataBase. The DataBase deletes the Asset and the Manager will notify the user of the activity being successful, after which it ends.

This activity diagram portrays another CRUD operation, in this case Deletion, in the system.

This helps getting an understanding of how deletion of objects works in the system, using Assets as an example. Deletion is crucial for the proper workings of the system.

Deletion is used with Assets, as seen on the diagram, but also with Results, where the user has the option to delete either of them.

4.3.6 UML Diagram - Activity Diagram (Optimize)

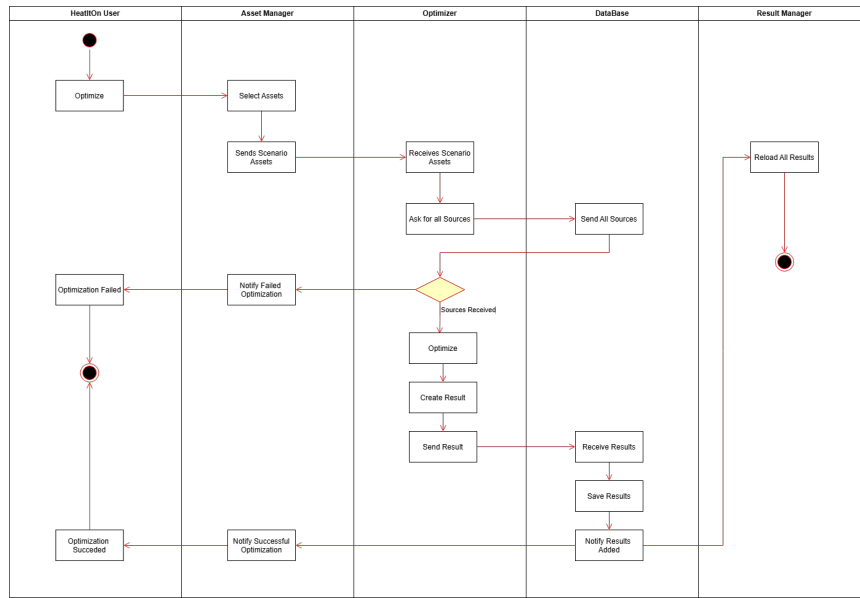


Figure 4.6: Optimization Activity UML Diagram - Three parts and their actions for deleting Assets.

The activity UML diagram (Figure 4.6) depicts the optimization process.

First, the optimization starts with the user, using the Asset Manager UI, selecting assets and starting the optimization process. The Asset Manager will send the selected Assets to the Optimization module, which after receiving them, will call from the Database all stored Sources up to that moment. If no Source is received it will prompt the Asset Manager to notify the user and end the activity. The Optimizer has no UI elements and is completely in the Backend. If it receives Sources, it will run the optimization algorithm, which will create a Result. The new Result is sent and stored in the Database that will notify both the Asset Manager(that in turn notifies the user) and the Result Manager that there is a new Result ready to be displayed, ending the activity.

This activity diagram proves its relevancy by demonstrating the optimization activity, which is the entire purpose of the system. While it is not enough for a full understanding of the optimization process, the diagram illustrates the different overall steps happening.

Optimization is the part that interacts with every module of the system. It uses directly or indirectly every implemented Manager, and all the activity diagrams (and all their depicted actions) are just helping the optimization process depicted here.

4.3.7 UML Diagram - Sequence Diagram

This later...

4.4 System Technical Architecture

For the system architecture, from a technical standpoint, C# was chosen as the base technology stack. This was chosen as its .NET 9.0 version is suitable for the development of this system with its simplicity, the familiarity of the team and its object-oriented imperative style. Every other piece of technology is related to .NET and was chosen based on their respective compatibility with the language.

Technology Stack

The first chosen piece was the DataBase that is running MySQL. The role of the DataBase is to store the system data independently from other parts of the system (Backend and Frontend). It stores relevant user data, using models, like the Assets, Sources, Results of each generator and Results of optimization runs. The DataBase communicates with the BackEnd using MicrosoftEntityFramework. The Frontend cannot communicate directly with the DataBase, it must do it through the BackEnd. The DataBase allows data storage that is independent from runs. The system can work with multiple users at the same time or with one user in different runs, storing all the data in the DataBase, where it will remain until deletion.

The second chosen piece of technology stack is for the FrontEnd UI. For this AvaloniaUI was chosen for its good compatibility with .NET and modularity. Besides this, the FrontEnd is following the Model-View-ViewModel(MVVM) design pattern.

Layering

The system is divided into three main parts, Frontend, Backend and the DataBase. Each serves a different role and, besides the DataBase, they are further separated for proper readability and modularity.

The FrontEnd, as stated earlier, follows the MVVM pattern. The Models, together with the Data (containing file I/O and clients) form the overall data and business layer. The View forms the presentation layer, containing all the UI elements. The ViewModel creates a connection layer between the two other parts.

The BackEnd is following a .NET web application template. Its role is to create a connection between the FrontEnd and DataBase with an Application Programming Interface (API), and to handle overall business logic for the entire system. For modularity, this is achieved through serious layering. The Backend consists of six main parts. Controllers handle incoming/outgoing traffic to/from the DataBase and FrontEnd and will call the appropriate Services. Services contain all the business logic of the system (like optimization or add asset). Both Controllers and Services inherit from two Interfaces, as they share similarities (like CRUD operations). The Data layer is composed of two other parts, Models (storing data models) and Data (storing DB Migrations and DB Context which is the main connection point to the DB). The presentation layer is completed by the last element, Hubs, which ensures real time communication with the FrontEnd application (instantly notifies to refresh the FrontEnd). Although the BackEnd is divided into many pieces, this was necessary for modularity, readability and because of complexity as well. This way each small section is responsible of one part, making replacement easier. In this solution, if one wants to hypothetically change the entire optimization algorithm, they only need to rework the Optimization Service, without having to touch any other parts.

Architectural Style

The project follows all major design principles. The main set of principles that the project adheres to, is SOLID. This happens in both FrontEnd and BackEnd and is visible in many parts, like interfaces or being able to add new features without having to change code. SOLID principles help create an Object-Oriented design. But besides these, the project follows DRY (Do not Repeat Yourself), KISS (Keep It Simple, Stupid), YAGNI (You Aren't Gonna Need It) and SoC (Separation of Concerns).

The technology stack, the layering and the architectural style used are all necessary for the proper running of the system. While the technology stack was kept simple by only using what was necessary for achieving requirements, the layering and style help build a good quality project that is easy to understand, change and complete. But extensive layering helps also following the style, by breaking up the large task into small sections creating a *Divide et impera* style of development, making it easier to adhere to the principles.

Chapter 5

Implementation and Technical Realization

HeatItOn is divided into three sections, and two versions. First there is the midterm evaluation demo version, and then there is the final version. We will go through both to see the first implementation and the improvements and changes that we made for the final version.

5.1 Frontend Implementation

Our Frontend Implementation is designed with the plant operators in mind, keeping it simple and user friendly. To achieve this, we started with a UI-sketch where we tried to come up with an easy to use and easy to understand concept, that we tried to achieve in our implementation.

The application is split into three different tabs. The first one, that the user sees when they launch the application is the Assets tab. Here the user can manage and see all the existing assets, they can import or export assets as csv's or manually add/remove or edit specific assets one by one. This page also has three preset toggles(Scenario 1, Scenario 2 and Custom). As the Assets page is where the operator will start the optimization process, by selecting the asset combination they want to optimize with, the toggles merely select from the assets the ones that are in that scenario. If they wish to optimize for a new scenario, they can select the Custom toggle, allowing them to run the optimization process for any scenario. After selecting the desired assets the user has to click on the Optimize button that will start the optimization process for the selected assets.

The list of assets is connected through the backend with the database, each modification being done live in the database. The optimization process from the backend is called from the frontend by clicking a button. The Assets tab will send the Optimization algorithm the assets selected to be optimized every time the button is clicked.

The second tab, the Results tab is where, once the user started an optimization, the result of that run will appear. On the left side, they can see and select out of a list of all results. When they select one result, the user can see the hourly result in a table format or view the data in different graphs. They can also export them into csv's. If the user would like to focus on a specific time period, not the entire optimization run, they can use a date-time picker to select from when to until when they want to see the result data, which will affect the result table and the graphs as well.

The tab loads all the optimization-results from the database using the backend services.

The third tab, Sources, is where the user will upload/export or even edit source data. On the left, you can see a list of uploaded csv's again. When the user selects one, they can also visualize the source data in a few simple charts or see the entire source data in a table. The cells of the table are modifiable so they can also edit the source data.

This tab uploads all sources directly to the database, using the backend services, while the csv file's are handled locally.

Our goal with each tab was achieving an easily navigable and understandable UI, while giving all the necessary tools for the plant-operators/users for adequate management. Although we only received source data for four weeks and two asset scenarios, we tried to create a scalable, modular UI, that can handle dozens of assets and years worth of source data. As a side goal we also wanted to achieve a simple yet modern and visually appealing look.

5.1.1 Midterm Version

For the midterm demo, we prioritised usability over aspect. As this version was a mere proof of concept, and its goal was to plant a picture of our way of implementing it and to show what we think is possible, some aspects of the overall user experience was ignored so we can focus on actual functionality.

This means that the UI for this version has a very generic, template look. Buttons are rough and placed where we could fit them, while icons are uninteresting and unintuitive. A majorly different approach for displaying assets, results and sources was used in this version. Instead of the frontend lists updating to changes accordingly(like uploading csv's or generating results) in real time, we added a refresh button that, when clicked, reloads all the data from the database. This solution was a compromise made for this version, and eliminating it, a focuspoint for the final version.

The Assets tab was fully functional, and only needed some visual and aesthetic improvements.

The Results tab in this version is defined by minimalistic graphs, that made it hard to visualize the results. Our three graphs(electricity use/production, heat production and CO2 production) were a good start but we realised we

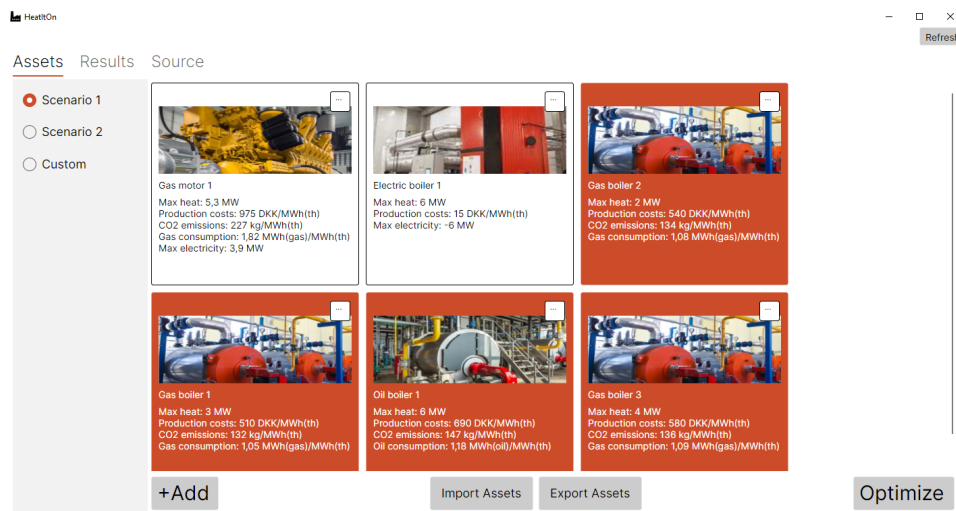


Figure 5.1: Assets tab from the demo version

need a lot more for proper visualization.

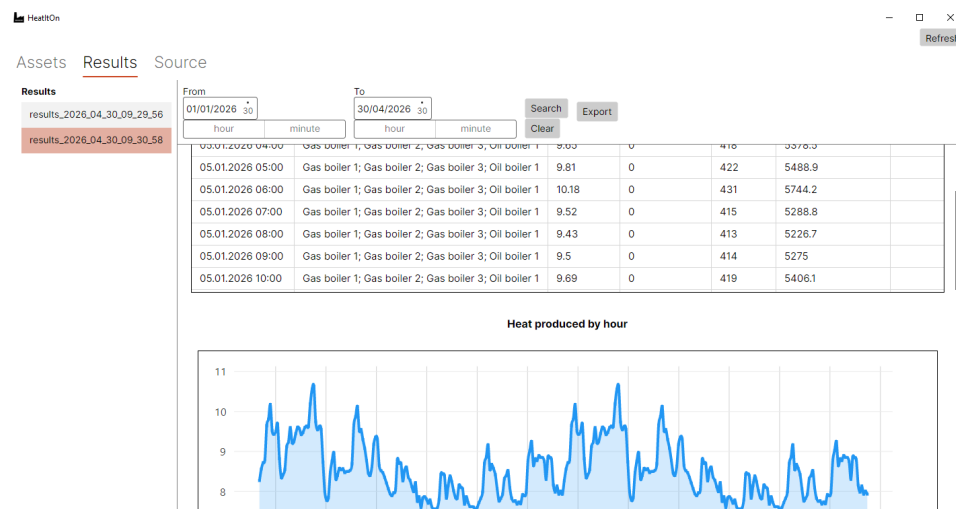


Figure 5.2: Results tab from the demo version

The Sources tab functioned as expected but the possibility to edit the csv data was missing from this version.

Overall the demo was a very solid proof of concept that helped us learn what we were missing for the final version while also showing us the direction we are going.

5.1.2 Final Version

5.2 Backend Implementation

5.2.1 Midterm Version

5.2.2 Final Version

5.3 Data Management

5.3.1 Midterm Version

5.3.2 Final Version

Chapter 6

Discussion and Reflection

6.1 Discussion and Reflection

This is our Discussion and Reflection.

Chapter 7

Conclusion

7.1 Conclusion

This is our Conclusion.

Chapter 8

References

8.1 References

This is our References.

Chapter 9

AI Usage

9.1 AI Usage

This is our AI Usage.

Appendix A

Appendix

A.1 Appendices

This is our Problem Statement.